

Python Mastery: Beautiful Soup

Web Scraping and Parsing HTML/XML Data

1. Introduction to Web Scraping

Web scraping is the process of extracting data from websites. **Beautiful Soup** is a Python library for pulling data out of HTML and XML files. It works with your favorite parser to provide idiomatic ways of navigating, searching, and modifying the parse tree.

Installing Requirements

You need to install the library and a request library (like `requests`) to fetch the webpage content.

```
pip install beautifulsoup4
pip install requests
```

2. Creating a Soup Object

The first step is to fetch the HTML content and pass it into the Beautiful Soup constructor along with a parser (usually `'html.parser'` or `'lxml'`).

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
page = requests.get(url)
soup = BeautifulSoup(page.content, 'html.parser')

# Prettify prints the HTML in a readable structure
print(soup.prettify())
```

3. Navigating the Data

Beautiful Soup provides several methods to navigate and search for specific tags.

Finding Tags

- `find()`: Returns the first occurrence of a tag.
- `find_all()`: Returns a list of all occurrences of a tag.

```
# Get the first <h1> tag
title = soup.find('h1').text

# Get all <a> tags (links)
links = soup.find_all('a')
for link in links:
    print(link.get('href'))
```

4. Searching by Attributes

You can search for tags with specific CSS classes, IDs, or other attributes.

```
# Find tag with a specific ID
element = soup.find(id="main-content")

# Find tags with a specific class (use class_ because 'class' is a keyword)
items = soup.find_all("div", class_="product-item")
```

5. Extracting Data

Once you've located a tag, you can extract its text or its attribute values.

```
tag = soup.find('p')
print(tag.text) # Returns the text inside the tag
print(tag.name) # Returns 'p'
print(tag['class']) # Returns the class attribute value
```

